

COMSATS University, Islamabad Pakistan

FalconScan
(AI-powered APK Vulnerability Scanner)

By

Mohamed Ahsan **CIIT/FA22-BCT-001/ISB**

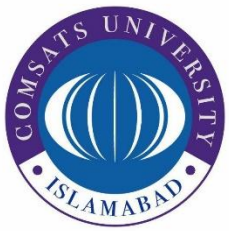
Abdullah **CIIT/FA22-BCT-004/ISB**

Supervisor

Dr. Farhana Jabeen

Bachelor of Science in Cyber Security (2022-2026)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.



COMSATS University, Islamabad Pakistan

FalconScan

(AI-powered APK Vulnerability Scanner)

A project presented to

COMSATS University, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Cyber Security (2022-2026)

By

Mohamed Ahsan **CIIT/FA22-BCT-001/ISB**

Abdullah **CIIT/FA22-BCT-004/ISB**

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other, we will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Mohamed Ahsan **Abdullah**

---------- ----------

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CT) FalconScan was developed by **Mohamed Ahsan (CIIT/FA22-BCT-001)** and **Abdullah (CIIT/FA22-BCT-004)** under the supervision of Dr. Farhana Jabeen and that in her opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Cyber Security.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

EXECUTIVE SUMMARY

With the rapid expansion of Android applications across industries, ensuring mobile app security has become a major challenge. Traditional vulnerability detection tools, such as MobSF and VirusTotal, primarily rely on static rule-based analysis, which is limited to identifying known threats. These approaches struggle to detect advanced or obfuscated vulnerabilities that exhibit malicious behavior only during runtime. Moreover, the manual interpretation of scan results consumes time and requires expert knowledge, making large-scale security testing inefficient.

To overcome these limitations, FalconScan has been developed, an AI-powered APK Vulnerability Scanner designed to automate the detection of both known and unknown security threats in Android applications. FalconScan integrates static and dynamic analysis techniques within a unified hybrid framework, providing comprehensive insights into app security.

The system performs static analysis by decompiling APKs and examining manifest files, permissions, and API usage, while its dynamic analysis engine executes the application in a controlled emulator environment to observe runtime behavior such as network communication, file access, and system calls. Extracted features from both phases are then analyzed using a Large Language Model (LLM), which intelligently classifies vulnerabilities and provides human-readable explanations for its decisions.

FalconScan operates through a web-based interface, allowing users to upload APKs, initiate scans, and view detailed vulnerability reports with severity levels and mitigation recommendations. The platform also maintains a **local vulnerability database** for quick rule-based detection and supports continuous learning through periodic AI model updates, ensuring adaptability to emerging threats.

By automating complex analysis workflows and incorporating AI reasoning, FalconScan significantly reduces manual effort, increases detection accuracy, and provides actionable insights for developers and cybersecurity professionals. It serves as an innovative, research-oriented, and scalable solution for modern Android application security assessment.

Abstract

The proposed project, FalconScan, is developed to create an intelligent and automated solution for detecting vulnerabilities in Android applications using AI-driven hybrid analysis. Existing systems such as MobSF and VirusTotal primarily rely on static rule-based checks and limited dynamic tracing, which fail to detect complex or obfuscated threats effectively. These traditional tools require significant manual interpretation and lack intelligent behavior analysis capabilities. The main challenge lies in identifying hidden or runtime-based vulnerabilities and reducing the dependence on fragmented tools and manual effort during mobile app security assessment. Current systems also lack adaptive learning and comprehensive behavioral insight. FalconScan aims to integrate static and dynamic analysis techniques with Large Language Model (LLM)-based reasoning to intelligently classify vulnerabilities, generate explainable reports, and support continuous AI model improvement. It will automate the scanning workflow, monitor runtime app behavior in an emulator, and provide detailed vulnerability reports through a web interface. The project also seeks to improve detection accuracy and scalability while minimizing manual intervention. Ultimately, FalconScan will offer a powerful, efficient, and intelligent solution that enhances mobile application security and assists developers and analysts in mitigating Android threats effectively.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Dr. Farhana Jabeen”. Without her personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are grateful to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us with the values of honesty & hard work.

Mohamed Ahsan

Abdullah

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
APK	Android Package Kit
AVD	Android Virtual Device
CI	Continuous Integration
CIIT	COMSATS Institute of Information Technology
CPU	Central Processing Unit
CVSS	Common Vulnerability Scoring System
DB	Database
DFA	Dynamic Fuzzing Analysis
DL	Deep Learning
FR	Functional Requirement
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HTTPS	Hypertext Transfer Protocol Secure
ISB	Islamabad Campus
JSON	JavaScript Object Notation
LLM	Large Language Model
MCP	Model Context Protocol
ML	Machine Learning
MobSF	Mobile Security Framework
NLP	Natural Language Processing
NFR	Non-Functional Requirement
OS	Operating System
PDF	Portable Document Format
PER	Performance Requirement
REL	Reliability Requirement
RQ	Requirement
SRS	Software Requirements Specification
SQL	Structured Query Language
TLS	Transport Layer Security
UI	User Interface
UML	Unified Modeling Language
USE	Usability Requirement
VM	Virtual Machine
VMS	Vulnerability Management System
VTS	VirusTotal Scanner
XML	Extensible Markup Language

Table of Contents

Abstract v

1 Chapter 1: Introduction and Problem Definition 3

1.1 Overview of the Project 3
1.2 Vision Statement 4
1.3 Problem Statement 4
1.4 Problem Solution 5
1.5 Objectives of the Proposed System 6
1.6 Scope 6
1.6.1 Limitations/Constraints 7
1.7 Modules 7
1.7.1 Module 1: APK Upload and Management 7
1.7.2 Module 2: Static and Fuzzing Analysis Engine 7
1.7.3 Module 3: Dynamic Analysis Engine 8
1.7.4 Module 4: LLM-Driven Vulnerability Analysis 8
1.7.5 Module 5: Reporting and Visualization 8
1.7.6 Module 6: User Interface and Access Control 8
1.8 Related System Analysis/Literature Review 9
1.8.1 Literature Review 9
1.8.2 Related System Analysis 9
1.9 Tools and Technologies 10
1.10 Project Contribution 11
1.11 Relevance to Course Modules 11

2 Chapter 2: Requirement Analysis 12

2.1 User Classes and Characteristics 12
2.1.1 Use case Diagram 13
2.1.2 Requirement Identifying Technique 13
2.2 Functional Requirements 15
2.2.1 Mockup M1 – Login Page 15
2.2.2 Mockup M2 – Dashboard Page 16
2.2.3 Mockup M3 – Upload APK Page 17
2.2.4 Mockup M4 – Scan Reports Page 18
2.2.5 Mockup M5 – Scan History Page 19
2.2.6 Event–Response Table for Backend Processes 20
2.3 Non-Functional Requirements 21
2.3.1 Reliability 21
2.3.2 Usability 21
2.3.3 Performance 21
2.3.4 Security 21
2.4 External Interface Requirements 22
2.4.1 User Interface Requirements 22
2.4.2 Software Interfaces 22
2.4.3 Hardware Interfaces 23
2.4.4 Communication Interfaces 23

3 Chapter 3: References 24

4 Chapter 4: Plagiarism Report 26

List of Figures

Figure 1: Use case diagram 13
Figure 2: Login page 15
Figure 3: Dashboard Page 16
Figure 4: Upload APK Page 17
Figure 5: Scan Reports Page 18
Figure 6: Scan Reports Page 19

1 Chapter 1: Introduction and Problem Definition

This chapter provides an overview of the proposed project FalconScan, explaining the background, motivation, objectives, and technical foundations of the system. It also outlines the scope, modules, and technological choices, as well as the project's contribution to the field of mobile application security and its alignment with relevant academic modules.

1.1 Overview of the Project

The FalconScan: AI-powered APK Vulnerability Scanner is an advanced web-based system that automatically detects vulnerabilities in Android application packages (APKs). The project belongs to the domain of mobile cybersecurity and artificial intelligence, combining static and dynamic analysis with large language model-based reasoning to identify and explain security weaknesses.

Android applications play an important role in almost every modern industry including finance, healthcare, education, and e-commerce. However, the open nature of the Android ecosystem makes it highly vulnerable to security threats such as malware injections, data leaks, unauthorized data access, and privilege escalation. Existing security scanners such as MobSF and VirusTotal mainly rely on rule-based and static analysis, which can only identify known threats. These traditional tools often fail to detect sophisticated, obfuscated, or zero-day vulnerabilities that emerge during runtime execution.

FalconScan introduces an intelligent and automated solution to overcome these challenges. It uses a hybrid security analysis framework that examines both the internal code structure and runtime behavior of APKs. Users such as developers, security analysts, and QA teams can upload APKs through a web interface. The system automatically analyzes them and generates detailed reports containing identified vulnerabilities, their severity levels, and recommended fixes.

The proposed system reduces the dependency on manual analysis, improves detection accuracy, and provides clear, human-readable explanations generated through artificial intelligence. FalconScan contributes to creating a safer Android application environment by combining automation, machine learning, and contextual understanding.

1.2 Vision Statement

For Android developers, cybersecurity researchers, and organizations seeking to ensure the security and reliability of their mobile applications, Who face increasing risks from malicious code, data breaches, and inefficient manual vulnerability testing, The FalconScan Is an intelligent, AI-driven vulnerability scanning system That automatically analyzes Android APK files using hybrid static-dynamic methods and LLM-based reasoning to detect, classify, and explain security vulnerabilities. Unlike traditional static analysis tools such as MobSF and VirusTotal that rely on predefined signatures and rule-based methods, Our product leverages machine learning and contextual understanding to detect both known and unknown vulnerabilities, providing actionable recommendations and adaptive threat intelligence.

1.3 Problem Statement

The security of Android applications remains a major concern due to the open-source nature of the Android ecosystem and the vast number of third-party apps distributed through both official and unofficial marketplaces. Traditional vulnerability scanners are often limited in their ability to detect dynamic threats, requiring expert-level manual intervention and interpretation of scan results. Developers frequently lack the tools and knowledge to identify issues such as permission misuse, insecure data storage, malicious API calls, or embedded malware behaviors during early development phases.

Moreover, static analysis alone cannot uncover vulnerabilities that manifest only during runtimes such as those involving network requests, dynamic code loading, or obfuscation techniques. Dynamic analysis tools exist but are resource-intensive and often produce unstructured output that is difficult to interpret without expert guidance. Consequently, there is a growing need for an automated, intelligent, and comprehensive platform that bridges these gaps.

1.4 Problem Solution

FalconScan is designed to overcome the limitations of existing tools by providing a hybrid, AI-enhanced APK vulnerability scanning system that combines static, dynamic, and AI-based reasoning techniques. The system performs static analysis by decompiling APKs to extract metadata, permissions, and API patterns, while the dynamic engine executes the application in a controlled environment to observe runtime behaviors such as file access, network communication, and system calls.

Using Large Language Model (LLM) integration, FalconScan can interpret and explain vulnerabilities in natural language, enabling even non-expert users to understand complex technical findings. The platform provides structured vulnerability reports that categorize findings by severity level and recommend mitigations.

Through this intelligent, automated, and user-centric approach, FalconScan minimizes manual effort, increases detection accuracy, and provides meaningful insights that empower developers to remediate threats effectively and enhance mobile application security across all stages of the software development lifecycle.

1.5 Objectives of the Proposed System

BO-1: Develop a hybrid static and dynamic analysis framework capable of detecting vulnerabilities in Android APKs.
BO-2: Integrate AI/LLM-based reasoning to classify vulnerabilities, provide detailed explanations, and suggest remediation actions.
BO-3: Automate the entire vulnerability scanning process through an intuitive web interface to minimize manual effort and human error.
BO-4: Maintain a local vulnerability database for quick rule-based detection and continuous learning.
BO-5: Enable dynamic behavior monitoring using an Android emulator for runtime threat detection.
BO-6: Generate comprehensive and readable security reports for developers, researchers, and organizations.
BO-7: Integrate fuzzing techniques within static analysis to automatically generate and test malformed or boundary-case inputs, detecting potential vulnerabilities in APK components before runtime.

1.6 Scope

The scope of FalconScan encompasses the complete cycle of Android application security assessment, from APK submission to final report generation. The system will allow users to upload APKs through a secure web portal, which will trigger automated analysis processes. The static analysis engine will inspect decompiled source files, manifest configurations, and permission structures, while the dynamic engine will simulate real execution scenarios using an emulator to capture runtime events and potential malicious activity.

The extracted features will then be processed by a machine learning classifier and an LLM-based interpretation engine, which together generate detailed, structured vulnerability reports. These reports will include vulnerability categories (e.g., insecure API calls, data leaks, privilege escalation), severity ratings, and textual explanations with remediation guidelines.

The system will be modular and extensible, allowing future integration of new scanning engines, threat models, and AI capabilities. The target users include Android developers, QA testers, cybersecurity professionals, and academic researchers in mobile security. FalconScan will support multiple concurrent analyses, secure file handling, and role-based access control for administrators and users.

Additionally, the project will focus on performance optimization, user experience, and scalability to accommodate large APKs and complex applications. The system will not include advanced malware reverse engineering, obfuscation removal, or real device testing in this version, but it will be architected to support these extensions in the future.

1.6.1 Limitations/Constraints

LI-1: FalconScan currently supports only Android applications in APK format; iOS or hybrid app formats are excluded.
LI-2: Real device dynamic testing is not included—dynamic analysis is performed within a virtualized emulator.
LI-3: The LLM reasoning is limited by the available training dataset and might occasionally generate partial or imprecise explanations.
LI-4: Internet connectivity is required for certain AI processing components and external vulnerability database updates.
LI-5: The system is intended primarily for research and educational use; commercial-scale integration may require additional scalability enhancements.

1.7 Modules

1.7.1 Module 1: APK Upload and Management

- **FE-1:** Upload APK files via the web interface for scanning.
- **FE-2:** Store uploaded APKs with metadata (name, size, hash, upload date).
- **FE-3:** Allow users to browse scan history and reports.
- **FE-4:** Delete or archive previously scanned APKs.

1.7.2 Module 2: Static and Fuzzing Analysis Engine

- **FE-1:** Decompile APKs using local tools (apktool, JADX).
- **FE-2:** Analyze AndroidManifest.xml for permission misuse, exported components, and intent filters.
- **FE-3:** Perform static fuzzing-based input mutation on extracted functions, configuration values, and intents to simulate edge-case inputs that may reveal hidden vulnerabilities.
- **FE-4:** Scan decompiled code for hardcoded credentials, insecure network configurations, risky API usage, and weak cryptography patterns.
- **FE-5:** Generate synthetic inputs using fuzzing algorithms (e.g., mutation-based and grammar-based fuzzers) to test the robustness of APIs, file handlers, and inter-component communications.
- **FE-6:** Record anomalies or exceptions triggered during fuzzing for correlation with dynamic analysis findings.
- **FE-7:** Extract structural features (permissions, intents, API dependencies, fuzzing crash indicators) for AI-driven classification.

1.7.3 Module 3: Dynamic Analysis Engine

- **FE-1:** Deploy APKs in an isolated Android emulator (AVD/Genymotion).
- **FE-2:** Monitor runtime behavior (file access, network connections, service invocations).
- **FE-3:** Capture logs, system calls, and runtime exceptions.
- **FE-4:** Perform controlled input injection to simulate user interaction.
- **FE-5:** Extract behavior features for AI processing.

1.7.4 Module 4: LLM-Driven Vulnerability Analysis

- **FE-1:** Preprocess extracted static and dynamic features.
- **FE-2:** Classify APKs as benign or malicious using trained LLM model.
- **FE-3:** Display vulnerability scores and AI confidence levels.
- **FE-4:** Update model with new training data periodically (offline learning module).
- **FE-5**: Utilize the **Model Context Protocol (MCP)** to structure input features, standardize analysis context, and enable traceable decision paths from AI responses.

1.7.5 Module 5: Reporting and Visualization

- **FE-1:** Generate detailed vulnerability reports with categorized issues.
- **FE-2:** Visualize dynamic behaviors (e.g., system call graphs, network maps).
- **FE-3:** Allow export reports as PDF or JSON.
- **FE-4:** Summarize overall APK risk score and provide actionable suggestions.

1.7.6 Module 6: User Interface and Access Control

- **FE-1:** Web-based dashboard for students, researchers, and developers.
- **FE-2:** Login and registration system with role-based access (admin/user).
- **FE-3:** Provide access control for scanning, viewing reports, and deleting data.
- **FE-4:** Audit logs of actions performed by each user.

1.8 Related System Analysis/Literature Review

1.8.1 Literature Review

Recent studies on Android vulnerability detection emphasize the importance of combining static and dynamic analysis for comprehensive security evaluation. Researchers such as *Faruki et al. (2022)* and *Wei et al. (2023)* highlight that static analysis alone cannot effectively detect runtime threats, while dynamic approaches face scalability and overhead challenges. Machine learning and deep learning have been increasingly explored to enhance detection accuracy and automate feature extraction from APKs.

However, most existing systems lack **interpretability**—they can flag a vulnerability but cannot explain *why* it exists or *how* to fix it. Furthermore, rule-based systems fail to detect new or obfuscated malware. LLM-based reasoning, a novel approach introduced through models like GPT and BERT, has demonstrated potential in understanding context, summarizing code behavior, and explaining vulnerabilities in natural language.

FalconScan advances this research by combining **traditional hybrid analysis** with **LLM explainability**, creating an intelligent, interpretable, and adaptive vulnerability scanner that addresses the key limitations of prior works.

1.8.2 Related System Analysis

Table 1. 1: Related System Analysis with proposed project solution

Application Name	Weakness	Proposed Project Solution
MobSF (Mobile Security Framework)	Limited to rule-based static/dynamic analysis; lacks AI reasoning and human-readable reports.	FalconScan integrates LLM reasoning to explain findings and identify novel threats.
VirusTotal	Focuses on signature-based malware detection, not vulnerability-level analysis.	FalconScan performs code-level hybrid analysis and behavior monitoring.
AndroBugs Framework	Offers limited dynamic analysis and lacks runtime behavior monitoring.	FalconScan adds sandbox-based dynamic behavior tracking.
QARK	Detects common security issues but lacks machine learning intelligence and contextual reporting.	FalconScan uses AI models for intelligent classification and detailed vulnerability context.

1.9 Tools and Technologies

Table 1. 2: Tools and Technologies for Proposed Project

Tools and Technologies	Purpose	Rationale
Python (Django Framework)	Backend development, API management	Robust, scalable, and widely used in AI-based web applications.
React.js	Frontend development	Fast, component-based UI for responsive and interactive dashboards.
TensorFlow / PyTorch	Machine learning model implementation	Supports training and deployment of AI models for vulnerability classification.
AFL (American Fuzzy Lop) / libFuzzer	Input mutation and fuzz testing for static analysis	Automates the generation of malformed inputs to uncover robustness issues during static scanning.
OpenAI API / LLM Integration	Natural language reasoning and explanation	Enables contextual analysis and explanation of vulnerabilities.
SQLite / PostgreSQL	Database management	Reliable and lightweight storage for user data and reports.
Android Emulator (AVD)	Dynamic analysis environment	Provides a safe, isolated environment for runtime app testing.
Docker	Application containerization	Ensures consistent deployment and environment management.

1.10 Project Contribution

The **FalconScan** system introduces several technical and conceptual contributions that enhance existing vulnerability detection methods:

- **AI-Enhanced Vulnerability Classification:** Integrates machine learning to detect unknown or obfuscated threats beyond rule-based detection.
- **LLM Explainability:** Uses large language models to generate natural language explanations and remediation guidance.
- **Hybrid Static-Dynamic Analysis:** Combines both approaches to achieve comprehensive coverage of vulnerabilities.
- **Automated Workflow:** Minimizes human effort by automating the entire scan-to-report pipeline.
- **Extensible Architecture:** Designed to incorporate future modules, datasets, and AI models.
- **Real-time Reporting:** Provides immediate feedback via a web dashboard and downloadable reports.

Impact:
By merging AI reasoning with hybrid vulnerability detection, FalconScan delivers an intelligent, adaptive, and developer-friendly security scanning solution, improving mobile app resilience, audit transparency, and operational efficiency.

1.11 Relevance to Course Modules

- **Software Engineering:** Applied full SDLC phases including requirement analysis, system design, and testing.
- **Artificial Intelligence / Machine Learning:** Integrated LLMs and ML classifiers for intelligent vulnerability detection.
- **Database Systems:** Designed structured storage for user, scan, and vulnerability data using PostgreSQL.
- **Web Technologies:** Built with React and Django for frontend and backend web services.
- **Cybersecurity:** Implemented secure authentication, sandboxed app execution, and data encryption techniques.
- **Cloud Computing / DevOps:** Utilized Docker for deployment and scalability management.

2 Chapter 2: Requirement Analysis

This chapter provides a detailed analysis of the requirements for the FalconScan system. The purpose of this chapter is to define and understand what the proposed software must accomplish in terms of both functionality and performance. It outlines the user classes and their interactions, the system's functional and non-functional requirements, and its external interface needs. This requirement analysis forms the foundation for subsequent design, development, and implementation phases of FalconScan – an AI-powered APK Vulnerability Scanner that intelligently analyzes Android applications for potential security risks through both static and dynamic analysis.

2.1 User Classes and Characteristics

FalconScan is designed for a variety of users ranging from developers to security professionals. Each user class interacts with the system differently depending on their role, responsibilities, and level of technical expertise. The following table defines each identified user class and its pertinent characteristics.

User Class	Description
Application Developer	The primary user who uploads Android APK files to scan for vulnerabilities. Developers use FalconScan to identify and fix potential threats such as insecure permissions, data leaks, or embedded malware. They have access to scan reports, vulnerability breakdowns, and AI-generated recommendations.
Security Analyst	A user who performs detailed analysis on scan reports and verifies AI-generated findings. Analysts may also use the system to test known malware or suspicious apps in a controlled environment to validate FalconScan's detection performance.
Administrator	The system administrator manages user accounts, controls system configurations, and monitors scanning operations and logs. Administrators have full privileges to update the AI model, manage API keys, and ensure smooth operation of the system.
Researcher	Users involved in cybersecurity research who use FalconScan's dataset and analytical outputs for vulnerability pattern studies. Researchers interact with scan results and statistical summaries but do not modify core system settings.
Guest User	A limited-access user who can perform demo scans or view general information about FalconScan without registration. This class is mainly intended for showcasing the platform's capabilities to potential users or evaluators.

2.1.1 Use case Diagram

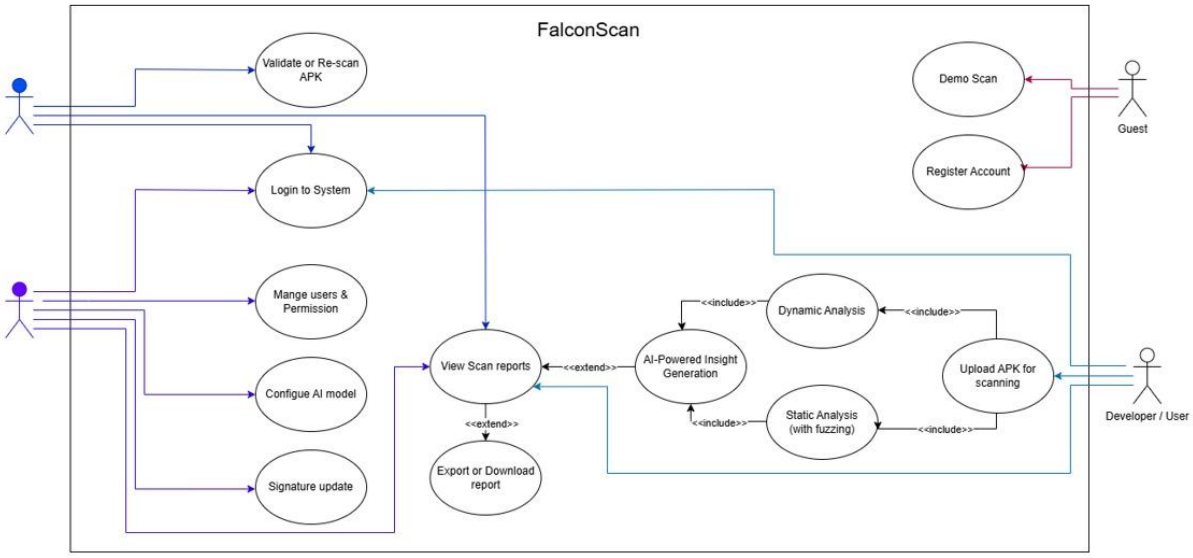


Figure 1: Use case diagram

2.1.2 Requirement Identifying Technique

For this project, **Mockup-Based Requirement Analysis** is used. This approach focuses on deriving functional requirements directly from the designed user interface mockups. Each mockup corresponds to a specific user interaction screen in FalconScan.

Steps Followed

1. Identified key features of FalconScan and represented each through a corresponding mockup.
2. Assigned Mockup IDs:
3. Created UI mockups that visualize each screen's layout and features.
4. Identified interactive elements such as buttons, upload fields, report cards, and filters.
5. Derived functional requirements based on the expected system behavior when these elements are used.
6. Organized all requirements using a tabular format (as shown in Section 2.2.1).

Backend Process Requirements

While the mockup-based technique effectively captures frontend behavior, FalconScan also includes extensive backend processes for file analysis, AI-based classification, and report generation.
For these, the Event–Response Table technique is used to define system behavior based on specific backend triggers and events.
Examples include:

- Handling file uploads and initiating analysis workflows.
- Running AI-based vulnerability detection models.
- Generating reports and storing historical scan data.

2.2 Functional Requirements

This section documents all the functional requirements derived from FalconScan's user interface mockups (M1–M5) and backend processes. Each requirement is defined clearly, testably, and traceably.

2.2.1 Mockup M1 – Login Page

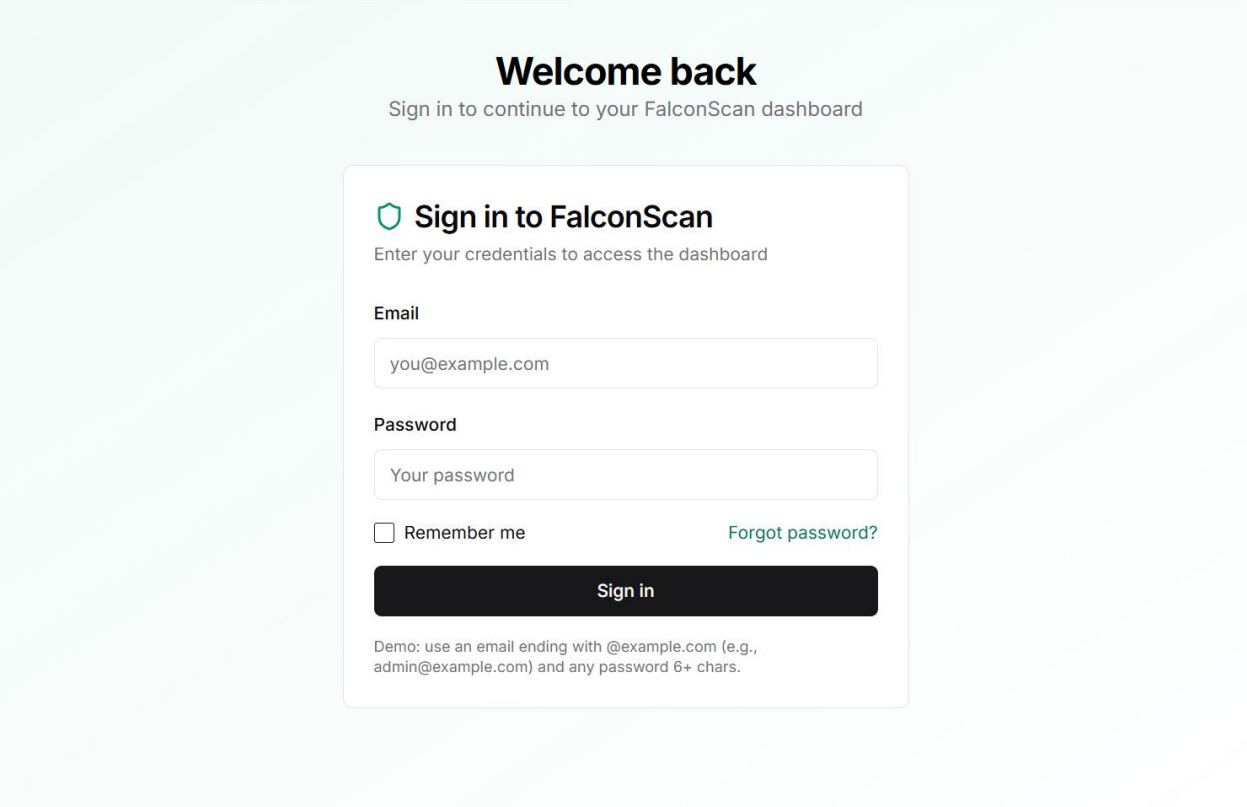


Figure 2: Login page

Functional Requirements Derived from Mockup M1 – Login Page

Table 2.2. 1: Functional Requirements Derived from Mockup M1

Feature (derived from UI)	Functional Requirement (FR-ID: Statement)	Business Rule
User Sign-In	FR1.1: The system shall allow users to log in using valid email and password credentials.	The email must follow the standard email format. The password must be at least 8 characters long.
User Authentication	FR1.2: The system shall validate user credentials against the registered database.	Accounts shall be locked after 5 failed login attempts.
Password	FR1.3: The system shall provide a	Only verified users can initiate

Recovery	"Forgot Password" option for registered users.	password recovery.
Session Management	FR1.4: The system shall maintain user sessions using a secure token-based method.	The session remains active for 30 minutes of inactivity.

2.2.2 Mockup M2 – Dashboard Page

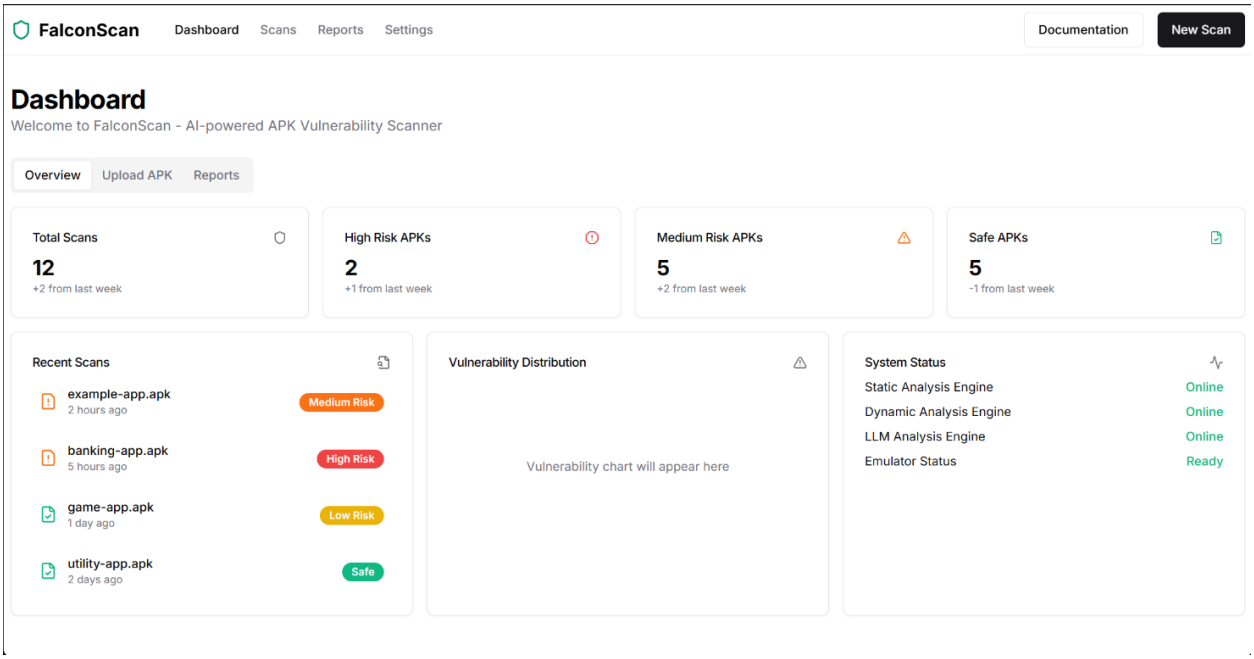


Figure 3: Dashboard Page

Functional Requirements Derived from Mockup M2 – Dashboard Page

Table 2.2. 2: Functional Requirements Derived from Mockup M2

Feature (derived from UI)	Functional Requirement (FR-ID: Statement)	Business Rule
User Overview	FR2.1: The system shall display user profile information and an overview of scan statistics.	Data should refresh in real time.
Navigation	FR2.2: The system shall provide navigation to Upload, Reports, and History pages.	Menu items depend on the user's access level.
Dashboard Widgets	FR2.3: The dashboard shall show summary widgets such as total scans, detected vulnerabilities, and last activity.	Widgets update dynamically from backend analytics.

2.2.3 Mockup M3 – Upload APK Page

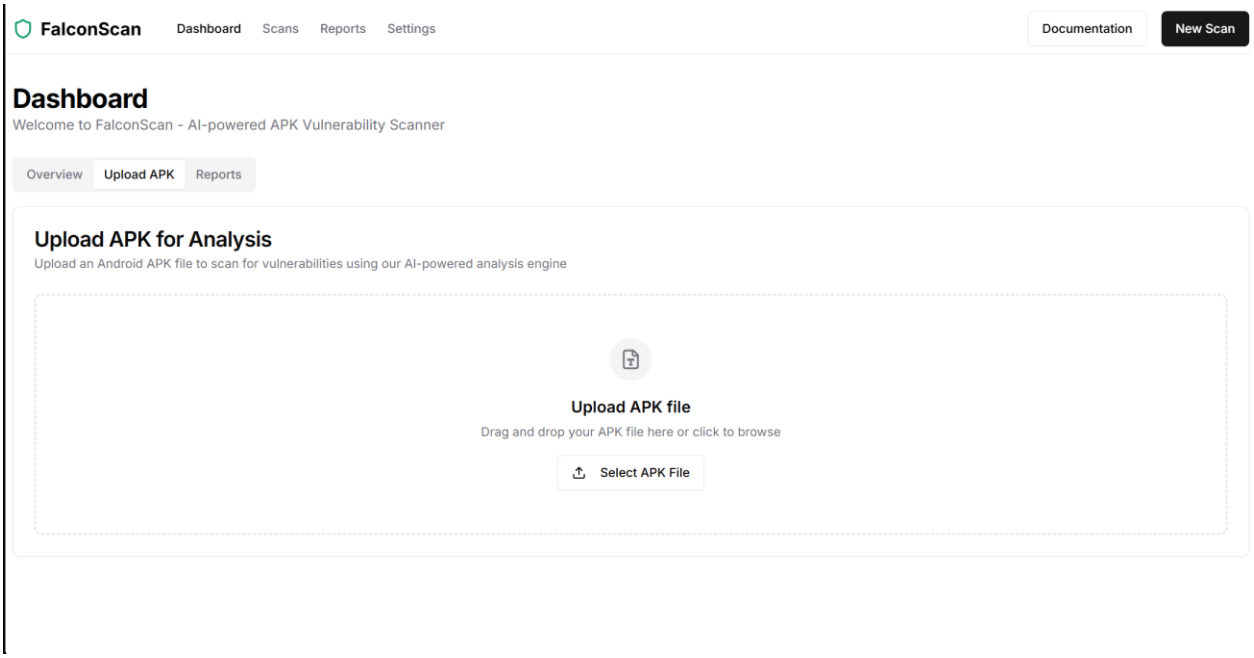


Figure 4: Upload APK Page

Functional Requirements Derived from Mockup M3 – Upload APK Page

Table 2.2. 3: Functional Requirements Derived from Mockup M3

Feature (derived from UI)	Functional Requirement (FR-ID: Statement)	Business Rule
APK Upload	FR3.1: The system shall allow users to upload Android APK files for analysis.	File size shall not exceed 200 MB.
Automated Analysis	FR3.2: The system shall perform static and dynamic analysis automatically upon upload.	The analysis is triggered once the upload completes.
Scan Progress Display	FR3.3: The system shall display scan progress in real time using progress bars.	Users can cancel ongoing scans before completion.
Access Control	FR3.4: The system shall restrict uploads to authenticated users only.	Guest users can only perform demo scans.
Fuzzing Configuration	Fuzzing Configuration	Fuzzing Configuration

2.2.4 Mockup M4 – Scan Reports Page

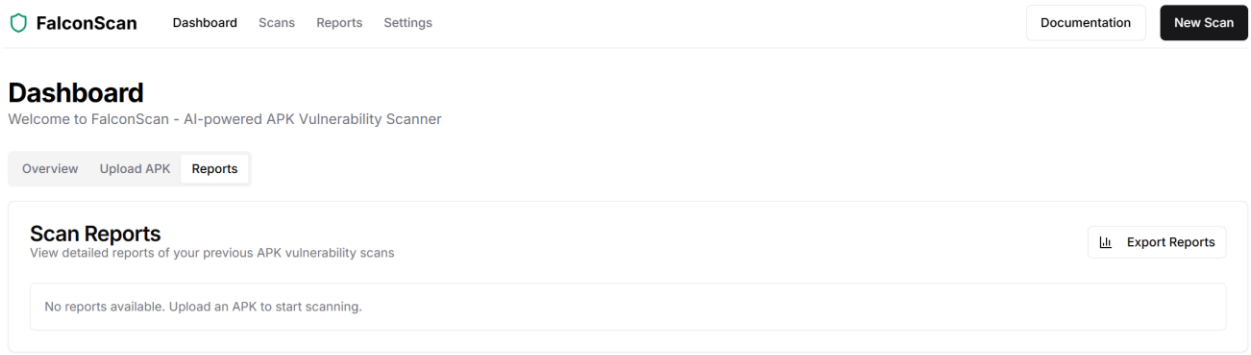


Figure 5: Scan Reports Page

Functional Requirements Derived from Mockup M4 – Scan Reports Page

Table 2.2. 4: Functional Requirements Derived from Mockup M4

Feature (derived from UI)	Functional Requirement (FR-ID: Statement)	Business Rule
Vulnerability Display	FR4.1: The system shall display a detailed report showing identified vulnerabilities categorized by severity (Critical, High, Medium, Low).	Vulnerabilities are ranked using CVSS standards.
AI Explanations	FR4.2: The system shall provide AI-generated explanations for each vulnerability.	Explanations must be concise and technical.
Report Export	FR4.3: The system shall allow exporting scan reports in PDF or JSON formats.	Only the scan owner can export reports.
Vulnerability Feedback	FR4.4: The system shall allow users to mark vulnerabilities as verified or false positive.	This feedback shall update the AI learning model.

2.2.5 Mockup M5 – Scan History Page

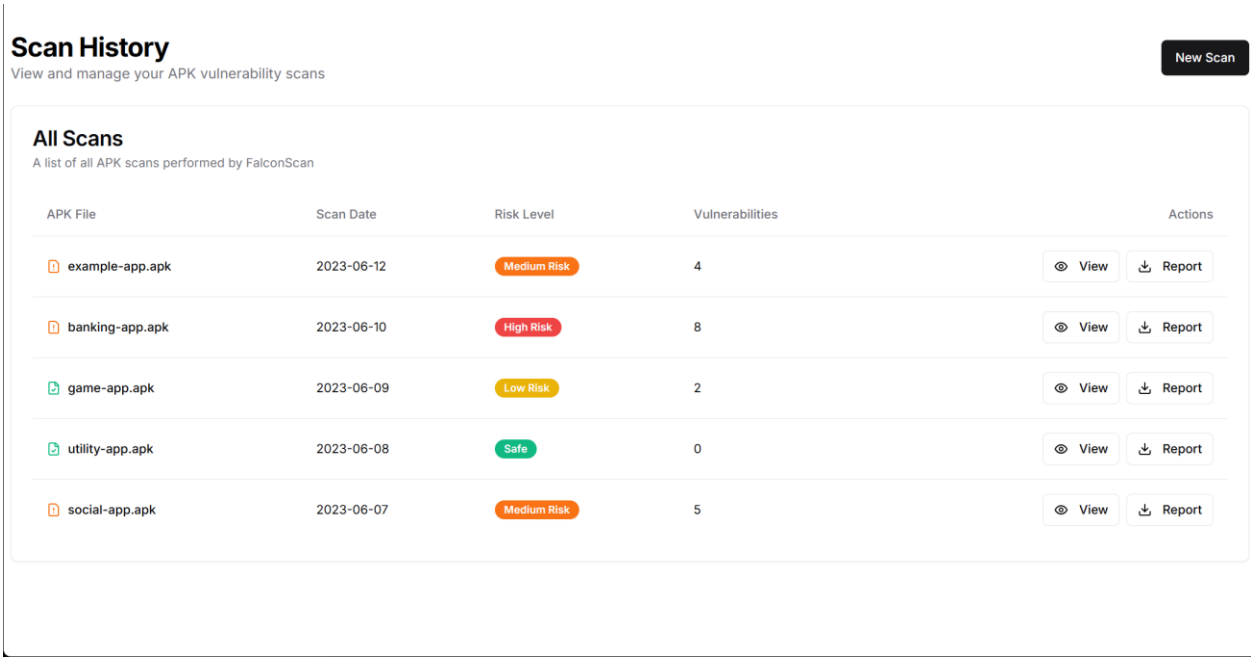


Figure 6: Scan Reports Page

Functional Requirements Derived from Mockup M5 – Scan History Page

Table 2.2. 5: Functional Requirements Derived from Mockup M5

Feature (derived from UI)	Functional Requirement (FR-ID: Statement)	Business Rule
Scan Records	FR5.1: The system shall maintain a chronological list of past scans for each user.	The list shall include date, filename, and scan status.
Filtering Options	FR5.2: The system shall allow filtering history by date range or scan result.	Maximum of 100 records displayed per page.
Re-scan Functionality	FR5.3: The system shall allow users to re-scan any previous APK from history.	Only the original uploader can re-scan files.

2.2.6 Event–Response Table for Backend Processes

Table 2.2. 6: Event–Response Table for FalconScan

Event	System State	Response
User uploads an APK file	File received, awaiting scan	The system stores the file temporarily and triggers analysis pipeline.
Static analyzer detects insecure permission	App under static analysis	The system records the permission issue and flags severity as Medium or High.
Dynamic analyzer detects malicious network request	App executing in sandbox	The system captures runtime behavior logs and stores them for reporting.
AI model completes classification	All analysis data processed	The system categorizes vulnerabilities, generates explanations, and stores results in the database.
User requests report export	Report generated	The system formats data into the chosen format (PDF/JSON) and initiates download.
Admin updates AI model	Model updated successfully	The system archives the old model version and applies the new one for future scans.
Fuzzing analysis initiated	Fuzzing analysis initiated	Fuzzing analysis initiated
Fuzzing detects abnormal function behavior	Fuzzing detects abnormal function behavior	Fuzzing detects abnormal function behavior

2.3 Non-Functional Requirements

2.3.1 Reliability

- **REL-1:** The system shall maintain a minimum uptime of 99.5% under normal operating conditions.
- **REL-2:** The system shall have a Mean Time Between Failures (MTBF) of at least 500 **hours**.
- **REL-3:** All scan progress shall be auto-saved every 30 seconds to prevent data loss during system failure.
- **REL-4:** In the event of an analysis crash, the system shall resume from the last saved state.

2.3.2 Usability

- **USE-1:** The system shall allow first-time users to complete a full scan process (upload → report) within 3 intuitive clicks**.**
- **USE-2:** The user interface shall maintain consistent color schemes, icons, and typography across all pages.
- **USE-3:** Tooltips and help sections shall be available for all major features.
- **USE-4:** The platform shall be accessible via both desktop and mobile browsers with responsive design.

2.3.3 Performance

- **PER-1:** 90% of scans for APKs under 100 MB shall complete within 3 minutes on standard cloud infrastructure.
- **PER-2:** 95% of pages shall load fully within 4 seconds over a 20 Mbps connection.
- **PER-3:** Concurrent scanning sessions shall support at least 50 simultaneous users without degradation.

2.3.4 Security

- **SEC-1:** The system shall use HTTPS for all communications between client and server.
- **SEC-2:** Uploaded files shall be isolated in sandboxed environments to prevent system compromise.
- **SEC-3:** All user data shall be encrypted in transit and at rest using AES-256.
- **SEC-4:** The system shall automatically log all access attempts and notify the admin of failed authentication exceeding 5 consecutive attempts.
- **SEC-5:** The AI model training data shall be anonymized and stored securely.

2.4 External Interface Requirements

2.4.1 User Interface Requirements

The FalconScan UI shall follow a clean, consistent design standard based on Material UI principles.
Key UI requirements include:

- Standardized color palette: dark theme with accent highlights.
- Font: Open Sans or Roboto, consistent across all screens.
- Every page shall include a top navigation bar with Home, Dashboard, Upload, Reports, and History links.
- Common buttons such as “Scan”, “Export”, and “Logout” will be present on applicable screens.
- All error messages will appear in red banners at the top of the interface with clear descriptions.
- A help icon (“?”) will appear on each screen for guidance.
- The interface shall support accessibility features such as screen reader compatibility and high-contrast mode.

2.4.2 Software Interfaces

Table 2.4. 1: Software Interfaces

Interface ID	Description
SI-1: AI Model API	The system shall communicate with a Python-based vulnerability detection API for AI processing.
SI-2: Database Interface	The system shall interact with a PostgreSQL database for user, scan, and report management.
SI-3: Emulator Service	The system shall interface with an Android emulator API to perform dynamic runtime analysis.
SI-4: Report Generator Module	The system shall generate and export reports via an integrated Python report generation library.

2.4.3 Hardware Interfaces

FalconScan runs primarily on cloud-based infrastructure but requires access to specific hardware during analysis:

- Android emulator host machines with at least 8 GB RAM and 4 CPU cores for dynamic testing.
- Server instances capable of GPU acceleration for AI model inference.
- Secure storage devices for maintaining uploaded APKs and scan results.

2.4.4 Communication Interfaces

Table 2.4. 2: Communication Interfaces

Communication ID	Requirement
CI-1:	The system shall communicate over HTTPS using TLS 1.3 protocol.
CI-2:	The system shall send email notifications to users upon completion of scans.
CI-3:	The system shall allow API communication for integration with third-party tools.
CI-4:	The system shall log all internal communications for traceability and debugging.

3 Chapter 3: References

1. *garak: A Framework for Security Probing Large Language Models*. Available at: https://github.com/NVIDIA/garak (Accessed: 5 May 2025).
2. *Assessing the Effectiveness of LLMs in Android Application Vulnerability Analysis*. Available at: https://arxiv.org/html/2406.18894v1 (Accessed: 12 April 2025).
3. *Android-Security-Assessment-Tools*. Available at: https://github.com/rostomzed/Android-Security-Assessment-Tools (Accessed: 22 April 2025).
4. *Assessing the Effectiveness of LLMs in Android Application Vulnerability Analysis*. ResearchGate. Available at: https://www.researchgate.net/publication/389809197_Assessing_the_Effectiveness_of_LLMs_in_Android_Application_Vulnerability_Analysis (Accessed: 30 April 2025).
5. *Android-Security-Assessment-Tools*. Available at: https://github.com/rostomzed/Android-Security-Assessment-Tools (Accessed: 17 May 2025).
6. *Vooki Android App Scanner*. Available at: https://www.vegabird.com/yaazhini/ (Accessed: 8 May 2025).
7. *Android Application Security Testing*. Available at: https://www.youtube.com/watch?v=PbvrxCvjsuY (Accessed: 2 May 2025).
8. *Android Penetration Testing with MobSF | OWASP ASST*. Available at: https://www.youtube.com/watch?v=qV_UolZEjuc (Accessed: 10 April 2025).
9. *Mobile App Testing and Security*. Available at: https://www.veracode.com/security/mobile-app-testing (Accessed: 25 April 2025).
10. *App Defense Alliance - MASA Requirements*. Available at: https://appdefensealliance.dev/masa/masa-requirements (Accessed: 19 May 2025).
11. *LLM Vulnerability Scanning Using Guardrails*. Available at: https://docs.nvidia.com/nemo/guardrails/latest/evaluation/llm-vulnerability-scanning.html (Accessed: 29 April 2025).
12. *Security in Mobile Applications: An Empirical Study on Android*. University of Padua. Available at: https://thesis.unipd.it/retrieve/56187322-2a56-4404-8537-626a447eb9ea/Braconaro_Elisa.pdf (Accessed: 14 April 2025).
13. *Vulnerability Scanners Topic*. Available at: https://github.com/topics/vulnerability-scanners (Accessed: 21 May 2025).
14. *SpringerOpen Article on Cybersecurity Topics*. Available at: https://cybersecurity.springeropen.com/articles/10.1186/s42400-025-00361-w (Accessed: 7 April 2025).
15. *Large Language Model (LLM) for Software Security Code Analysis, Malware Analysis & Reverse Engineering*. ResearchGate. Available at: https://www.researchgate.net/publication/390671720_Large_Language_Model_LLM_for_Software_Security_Code_Analysis_Malware_Analysis_Reverse_Engineering (Accessed: 27 May 2025).
16. *Electronics Journal - Recent Trends in Electronics and Electrical Engineering*. Available at: https://www.mdpi.com/2079-9292/13/23/4718 (Accessed: 3 May 2025).
17. *Vulnerability Detection in Recent Android Apps: An Empirical Study*. ResearchGate. Available at: https://www.researchgate.net/publication/345162631_Vulnerability_detection_in_recent_Android_apps_An_empirical_study (Accessed: 18 April 2025).
18. *Advances in Software Security and Machine Learning*. Available at: https://arxiv.org/html/2405.03644v2 (Accessed: 20 May 2025).
19. *garak: A Framework for Security Probing Large Language Models*. Available at: https://github.com/NVIDIA/garak (Accessed: 11 May 2025).
20. *Assessing the Effectiveness of LLMs in Android Application Vulnerability Analysis*. Available at: https://arxiv.org/html/2406.18894v1 (Accessed: 9 April 2025).

21. *Android-Security-Assessment-Tools*. Available at: https://github.com/rostomzed/Android-Security-Assessment-Tools (Accessed: 24 April 2025).
22. *Understanding the Impact of LLMs in Cybersecurity and Software Engineering*. arXiv preprint arXiv:2502.18474. Available at: https://arxiv.org/abs/2502.18474 (Accessed: 16 May 2025).

4 Chapter 4: Plagiarism Report

5% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

5 AI-generated only 0%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.